

Prediction and Control by Dynamic Programming



Dynamic Programming for MDPs – Big Picture

Goal: Solve Markov Decision Processes (MDPs) with a known model

Use Bellman equations as recursive definitions

Two core tasks: policy evaluation and control

Dynamic Programming (DP) = planning with a perfect model

Algorithms: policy iteration and value iteration

Planning in MDPs – Definition and Formulation

Planning: compute good behavior before acting

Input: full MDP model (S, A, P, R, λ)

Output: a (near-)optimal policy π

Objective: maximize expected return G_t

Uses: robotics, operations research, game AI with known rules



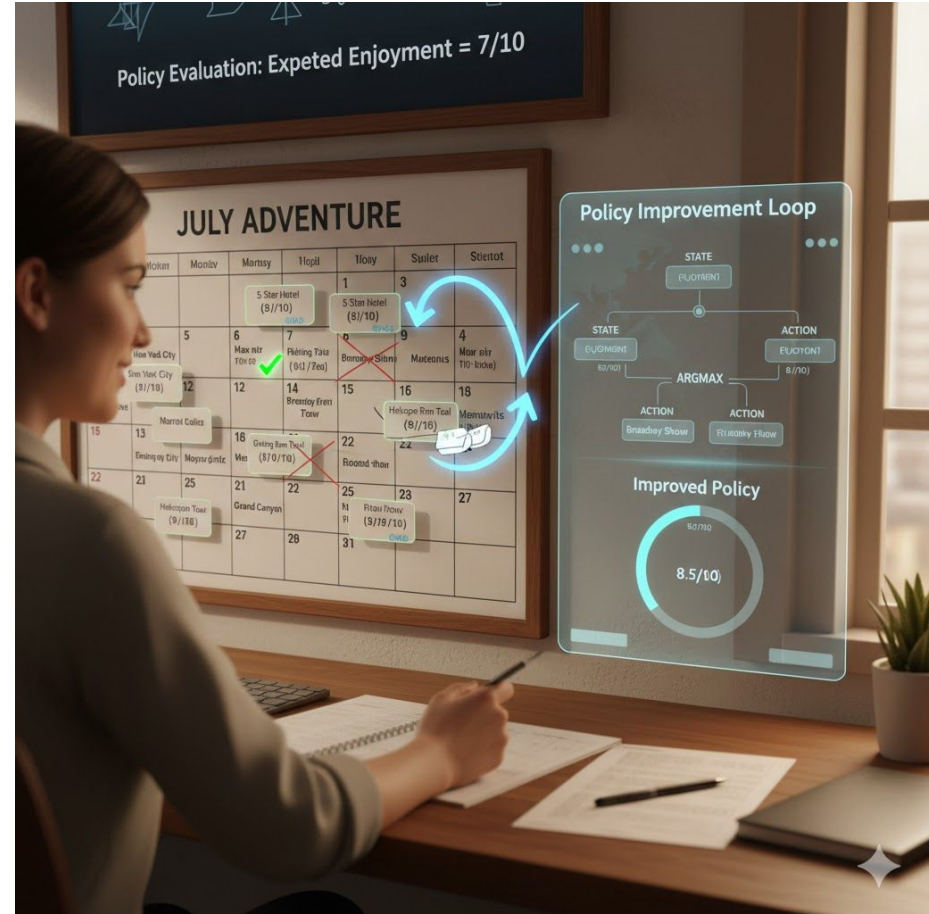
Policy Evaluation (Prediction)

- Task: compute $V^\pi(s)$ for a given policy π
- Bellman expectation equation:
 - $$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')]$$
- Iterative policy evaluation:
 - Initialize $V_0(s)$ arbitrarily
 - Repeatedly apply Bellman updates to all states
- Stop when value changes are below threshold



Policy Improvement

- Given $V^\pi(s)$, define a better policy π'
- Greedy improvement:
 - $\pi'(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')]$
- Policy Improvement Theorem: π' is at least as good as π
- Repeat improvement until policy stops changing



Policy Iteration

- Combines policy evaluation + policy improvement
- Algorithm:
 1. Initialize π_0 arbitrarily
 2. **Policy evaluation** $\rightarrow V^{\pi_k}$
 3. **Policy improvement** $\rightarrow \pi_{k+1}$ from V^{π_k}
 4. Repeat until $\pi_{k+1} = \pi_k$
- Guaranteed to converge to an optimal policy for finite MDPs

Value Iteration

- Integrates evaluation and improvement into one update

- Bellman optimality update:

- $V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V_k(s')]$

- Initialize $V_0(s)$ arbitrarily

- Iterate Bellman optimality backups until convergence

- Extract greedy policy: $\pi^*(s) = \arg \max_a \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V^*(s')]$

Efficiency of Dynamic Programming

- Time per sweep: $O(|\mathcal{S}|^2|\mathcal{A}|)$ in tabular case
- Requires full sweeps over all states (often many times)
- Needs complete model P, R (planning, not learning)
- Suffers from “curse of dimensionality”
- Foundation for approximate and sample-based methods